

TRAMA · MANI — Documento de Herramientas, Políticas y Lineamientos V2

Proyecto: MANI — Plataforma Multi-Tenant de Formalización y Gestión de Servicios

Organización: TRAMA · Ingeniería de Software

Documento: Herramientas, Políticas y Lineamientos V2 (Entregable Oficial Sprint 1 Review & Planning Sprint 2)

Versión: 2.0 (Línea Base para Sprint 2)

Fecha: Septiembre 2026

Responsables: Sara Albarracín (Scrum Master), Daniel Ávila (DevOps Titular), Nicolás León (Product Owner), Santiago (QA / Security Testing) y Mesa de Arquitectura.

Fuentes y Trazabilidad: ADR-0001 a ADR-0017 (Repositorio `Trama-AS/MANI-docs`), SAD V1, SRS V2, Matriz de Herramientas V1, Gobierno del Equipo.

Control de Versiones del Documento

Versión	Fecha	Autor(es)	Descripción del Cambio	Estado
1.0	2026-08-20	Sara Albarracín, Daniel Ávila	Línea base preliminar de herramientas de proceso y desarrollo (Entrega 2 / Sprint 0). Múltiples tecnologías en estado "Evaluar" o pendientes de decisión.	Superada
2.0	2026-09-02	Daniel Ávila, Santiago, Camila Beltrán, Sara Albarracín, Nicolás León	Consolidación integral V2. Se formaliza el Tech Radar V2 (ADR-0010), se incorpora la Arquitectura Tecnológica de Alto Nivel y Flujo de Comunicación (Jira ↔ GitHub ↔ CI/CD ↔ Ambientes ↔ DevSecOps ↔ Observabilidad ↔ Jira Feedback Loop), se ratifican las políticas de DevSecOps (ADR-0005, ADR-0013, ADR-0015), Observabilidad (ADR-0006), Gobernanza de IA (ADR-0009), y se resuelven las incompatibilidades de persistencia y stack técnico (ADR-0011, ADR-0012).	Aprobada (Línea Base Sprint 2)

0. Propósito y Reglas de Gobierno

El presente documento constituye la especificación vinculante de las herramientas, tecnologías, políticas técnicas y lineamientos de ingeniería que rigen el desarrollo, aseguramiento de calidad, seguridad de la información y operaciones de la plataforma **MANI** durante el **Sprint 2** y subsecuentes.

Reglas de Uso y Vigencia

- Lo que no está aquí, no se exige:** Ninguna validación, criterio de rechazo en Pull Requests ni evaluación de entregables puede fundamentarse en lineamientos que no consten explícitamente en este documento o en un ADR ratificado.
- Lo que está aquí, se cumple:** Las excepciones operativas o técnicas deben someterse a la Mesa de Arquitectura o dejarse registradas con responsable, justificación y fecha en el acta de la ceremonia correspondiente.
- Mecanismo de modificación:** Toda actualización a este documento requiere acuerdo en retrospectiva de sprint o ratificación formal en la Mesa de Arquitectura (quórum mínimo 5 de 7 integrantes) y entra en vigencia a partir del sprint siguiente.

1. Arquitectura Tecnológica de Alto Nivel y Comunicación entre Herramientas

Para garantizar la entrega continua, la trazabilidad sin fisuras y la seguridad por diseño (*Security by Design*), el proyecto MANI implementa un ecosistema de herramientas interconectadas que abarcan desde la concepción de requerimientos hasta la operación y monitoreo en tiempo real.

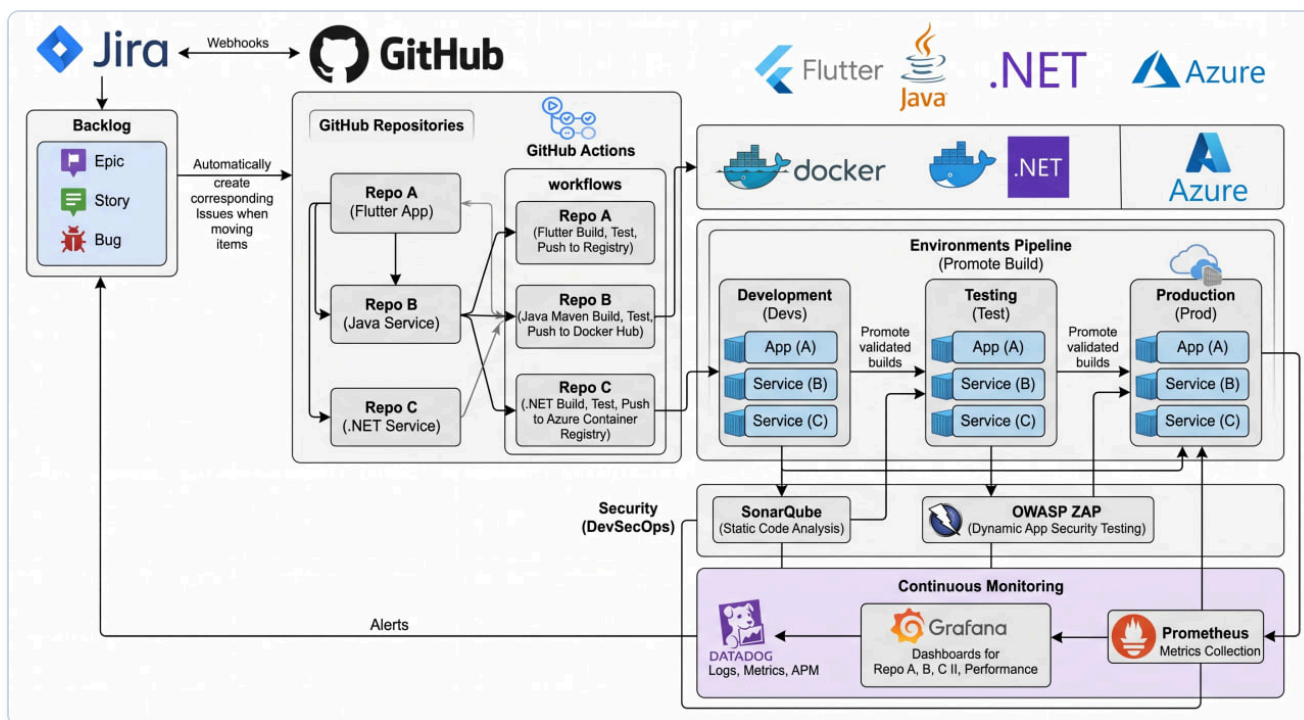


Figura 1.1: Diagrama de Tecnologías de Alto Nivel — Flujo de integración y comunicación entre herramientas del ciclo DevSecOps de MANI.

1.1 Ciclo de Gestión y Trazabilidad Bidireccional (Jira ↔ GitHub)

- **Rastreador y Backlog Unificado:** Conforme a **ADR-0002**, Jira es la herramienta única y oficial para la gestión ágil del proyecto, administración del Product Backlog (Épicas, Historias de Usuario, Tareas y Bugs) y control de tableros Kanban/Scrum para el Product Owner (PO) y Scrum Master (SM).
- **Sincronización vía Webhooks:** Se establece un canal de integración bidireccional mediante Webhooks entre Jira y la organización de GitHub.
- **Creación Automatizada de Issues y Ramas:** La transición de ítems en los tableros de Jira (por ejemplo, de *To Do* a *In Progress*) dispara automáticamente la generación de los Issues técnicos correspondientes en los repositorios de GitHub y habilita la creación de ramas normalizadas bajo la convención `feature/US-XX-descripcion` o `fix/BUG-XX-descripcion`.

1.2 Estructura Multi-Repositoryo

Siguiendo **ADR-0004**, MANI adopta un modelo desacoplado de múltiples repositorios especializados, evitando cuellos de botella de integración y respetando las naturalezas tecnológicas del producto: - **Repo A (mani-app-flutter):** Código fuente de la aplicación cliente y portal de aliados desarrollado en **Flutter / Dart**. - **Repo B (mani-backend-java):** Microservicio backend implementado en **Java** con gestión de dependencias y construcción en **Maven**. Empaqueta imágenes Docker y publica en Docker Hub / Container Registry. - **Repo C (mani-backend-dotnet):** Microservicio backend implementado en **.NET (C#)**. Automatiza su compilación, pruebas y publicación hacia Azure Container Registry (ACR).

1.3 Workflows de Integración Continua (GitHub Actions)

Cada repositorio posee flujos de trabajo (*workflows*) declarativos e independientes en GitHub Actions: - **Compilación y Pruebas:** Ejecución obligatoria de pruebas unitarias y de integración en cada *Push* y *Pull Request*. - **Empaquetado de Contenedores:** Construcción automatizada de imágenes Docker reproducibles e inmutables (*Build Once, Deploy Anywhere*). - **Publicación en Registros:** Envío controlado a Docker Hub (Repo B) y Azure Container Registry (Repo C y Repo A).

1.4 Pipeline de Promoción de Ambientes (Promote Build)

El despliegue hacia infraestructura cloud en **Microsoft Azure** se estructura en tres entornos secuenciales estrictamente segregados (**ADR-0004**): 1. **Development (Devs / rama develop):** Ambiente de integración continua para desarrolladores. Despliegue automático de las imágenes contenerizadas validadas por CI. 2. **Testing (Test / rama release):** Ambiente de pruebas de QA. Recepción de builds promovidos tras superar las pruebas unitarias y de integración. En este entorno se ejecutan las pruebas de contrato (Newman/Postman), pruebas de carga (k6) y pruebas dinámicas de seguridad (DAST). 3. **Production (Prod / rama main):** Entorno productivo de alta disponibilidad alojado en Azure. Ningún artefacto llega a producción sin haber sido promovido y validado exitosamente en Testing. La liberación requiere aprobación manual (*Environment Protection Rules*) por parte del DevOps titular. - **Orquestación con Kubernetes:** Conforme al requerimiento curricular y de arquitectura (PROY-08 / ADR-0004), los contenedores en ambientes superiores se orquestan mediante **Kubernetes** (AKS en Azure), garantizando escalabilidad y alta disponibilidad.

1.5 DevSecOps: Seguridad Continua (SAST y DAST)

De acuerdo con **ADR-0005**, la seguridad se integra de forma nativa en el flujo de entrega: - **SAST (Static Application Security Testing) con SonarQube:** Inspección estática del código fuente en las ramas de desarrollo y en cada Pull Request. Bloquea de manera vinculante la mezcla de código si se detectan vulnerabilidades de severidad *Blocker* o *Critical*. - **DAST (Dynamic Application Security Testing) con OWASP ZAP:** Pruebas dinámicas ejecutadas sobre los servicios web y endpoints activos en el ambiente de Testing. Detecta vulnerabilidades en tiempo de ejecución (inyección SQL, XSS, headers inseguros, fallos de autenticación) previo a autorizar el paso a Producción.

1.6 Observabilidad Continua y Bucle de Retroalimentación de Incidentes

Conforme a **ADR-0006**, la operación del sistema cuenta con telemetría integral y cierre del ciclo de retroalimentación hacia la gestión del proyecto: - **Prometheus:** Recolección continua de métricas a nivel de infraestructura, clúster y endpoints de microservicios (latencias, throughput, uso de recursos). - **Grafana:** Cuadros de mando y tableros centralizados para la visualización gráfica de rendimiento por repositorio (Repo A, Repo B, Repo C) y métricas globales. - **Datadog:** Agregación centralizada de logs estructurados en JSON, monitoreo de rendimiento de aplicaciones (APM) y trazabilidad distribuida entre microservicios. - **Cierre del Bucle (Feedback Loop cerrado hacia Jira):** Ante la detección de anomalías operacionales o incidentes críticos en producción, Datadog despacha automáticamente alertas que generan issues de tipo **Bug/Incidente** directamente en el Backlog de Jira, priorizando su atención inmediata por el equipo.

2. Tech Radar V2 del Proyecto MANI

El **Tech Radar V2** consolida y oficializa el portafolio tecnológico de MANI. Conforme a lo establecido en **ADR-0010**, se adopta esta estructura para brindar una visión global del stack, orientar el diseño arquitectónico y prevenir decisiones aisladas o informales.

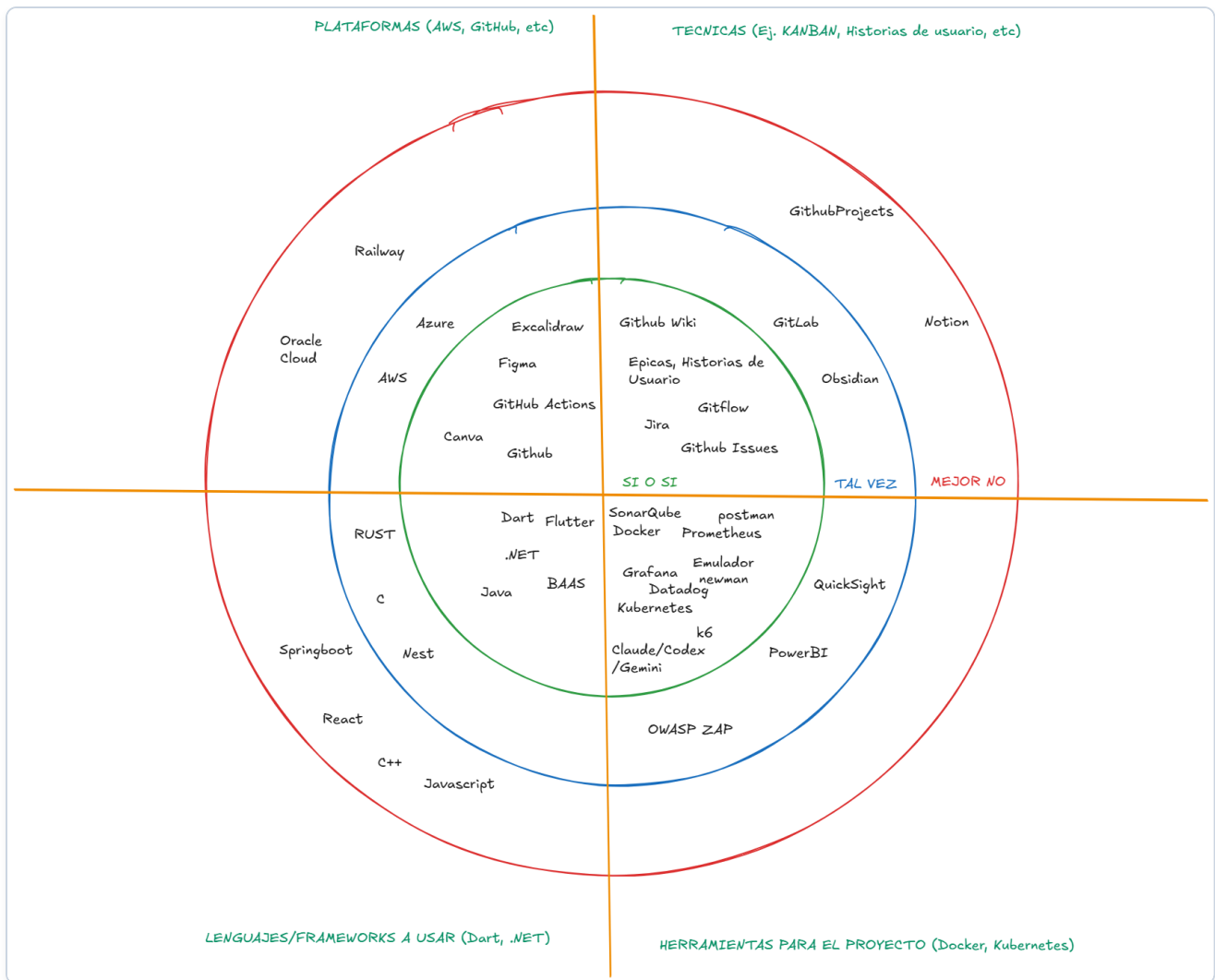


Figura 2.1: Tech Radar V2 — Clasificación de tecnologías, plataformas, técnicas y herramientas del proyecto MANI.

2.1 Metodología y Anillos de Confianza

El Tech Radar se estructura en tres anillos concéntricos que representan el nivel de confianza y madurez en el proyecto: - **● SÍ O SÍ (Adoptar / En Producción):** Tecnologías ratificadas por ADR, con decisiones arquitectónicas aprobadas, de uso obligatorio y plenamente integradas en los flujos del equipo. - **● TAL VEZ (Probar / Evaluar / En Espera):** Tecnologías viables, en etapa de prueba de concepto (*spike*), o condicionadas a necesidades de escala, presupuesto o requerimientos futuros. - **● MEJOR NO (Descartado / Contener):** Herramientas y tecnologías que han sido formalmente descartadas tras evaluación de trade-offs, incompatibilidad técnica con los drivers de calidad o exceso de sobrecarga operativa.

2.2 Detalle por Cuadrantes del Tech Radar

Cuadrante I: Plataformas (Cloud, Infraestructura y Diseño)

Tecnología	Anillo	Justificación y Criterio en MANI	ADR Relacionado
GitHub	SÍ O SÍ	Plataforma central para alojamiento de código fuente, gestión de Pull Requests, control de versiones y repositorio oficial de documentación técnica y ADRs en Markdown.	ADR-0001, ADR-0007
GitHub Actions	SÍ O SÍ	Motor oficial de integración y despliegue continuo (CI/CD). Automatiza pipelines de compilación, ejecución de suites de pruebas y publicación de imágenes.	ADR-0004
Figma	SÍ O SÍ	Herramienta estándar para diseño de experiencia de usuario (UX/UI), prototipado de alta fidelidad y mockups interactivos de la aplicación móvil y portales.	ADR-0008, SAD
Excalidraw	SÍ O SÍ	Plataforma para diagramación ágil, bocetos conceptuales y sesiones colaborativas de arquitectura en la Mesa de Arquitectura.	ADR-0008
Canva	SÍ O SÍ	Plataforma para diseño de activos gráficos corporativos, presentaciones institucionales e infografías de entrega.	—
Microsoft Azure	TAL VEZ	Infraestructura Cloud de referencia para el alojamiento de ambientes (Devs/Test/Prod), Azure Container Registry y servicios gestionados. Permanece en "Tal vez" para dimensionar costos frente a la capa gratuita/académica.	ADR-0004, ADR-0010
AWS	TAL VEZ	Proveedor cloud alternativo evaluado como plan de contingencia frente a límites de suscripción en Azure.	ADR-0010
Railway	TAL VEZ	Plataforma PaaS ágil evaluada para despliegues temporales de prototipos rápidos y pruebas tempranas de concepto.	ADR-0010
Oracle Cloud	MEJOR NO	Descartado por complejidad innecesaria en la curva de configuración de redes y cuotas frente a las necesidades del proyecto.	ADR-0010

Cuadrante II: Técnicas (Metodologías, Gobierno y Prácticas)

Técnica	Anillo	Justificación y Criterio en MANI	ADR Relacionado
Jira	SÍ O SÍ	Herramienta única de gestión ágil. Administra el Backlog, sprints, historias de usuario, estimaciones en puntos y seguimiento de métricas del equipo.	ADR-0002
Épicas e Historias de Usuario (BDD)	SÍ O SÍ	Técnica estándar para especificación de requerimientos funcionales, estructurados con criterios de aceptación en formato Gherkin / BDD (Dado / Cuando / Entonces).	SRS V2, SAD
Gitflow	SÍ O SÍ	Estrategia de ramas obligatoria para el control de versiones: <code>main</code> , <code>develop</code> , <code>release/*</code> , <code>feature/*</code> , <code>fix/*</code> , <code>spike/*</code> .	ADR-0004
GitHub Issues	SÍ O SÍ	Trazabilidad técnica de bajo nivel vinculada a Pull Requests, commits y disparada por transiciones de Jira vía webhooks.	ADR-0004
GitHub Wiki	SÍ O SÍ	Espacio complementario dentro de los repositorios para guías de instalación rápida (<i>Getting Started</i>) y documentación de referencia técnica.	ADR-0007
GitLab	TAL VEZ	Evaluated en fase inicial; se mantiene como alternativa teórica pero no se adopta para evitar duplicidad frente a GitHub.	ADR-0002, ADR-0010
Obsidian	TAL VEZ	Uso personal permitido para toma de notas de ingeniería local en Markdown, sin valor como repositorio oficial.	ADR-0001, ADR-0007
GitHub Projects	TAL VEZ	Evaluated para seguimiento puramente técnico interno; descartado para la gestión del producto para no competir con Jira.	ADR-0002, ADR-0010
Notion	MEJOR NO	Descartado para la documentación y gestión del proyecto para evitar dispersión documental fuera de GitHub y OneDrive.	ADR-0001, ADR-0007

Cuadrante III: Lenguajes y Frameworks

Lenguaje / Framework	Anillo	Justificación y Criterio en MANI	ADR Relacionado
Dart	SÍ O SÍ	Lenguaje primario adoptado tanto para la aplicación cliente multiplataforma como para el ecosistema móvil y backend Serverpod.	ADR-0010, ADR-0012
Flutter	SÍ O SÍ	Framework multiplataforma oficial para la construcción de la aplicación cliente y la aplicación de aliados (Repo A).	ADR-0004, ADR-0010
.NET (C#)	SÍ O SÍ	Requerimiento de proyecto (PROY-07). Microservicio de backend (Repo C) para lógica de alta concurrencia y contratos transaccionales.	ADR-0004, ADR-0010
Java (Maven)	SÍ O SÍ	Requerimiento de proyecto (PROY-07). Microservicio de backend (Repo B) para lógica de negocio empresarial empaquetada con Maven.	ADR-0004, ADR-0010
BaaS (Supabase / Serverpod)	SÍ O SÍ	Backend-as-a-Service sobre PostgreSQL gestionado, proveyendo Row-Level Security (RLS) para aislamiento multi-tenant estricto (RNF-01), autenticación y Supabase Storage para documentos KYC.	ADR-0012, ADR-0013, ADR-0017
Rust	TAL VEZ	Lenguaje evaluado para módulos críticos de despacho o criptografía en caso de requerirse optimización extrema de memoria.	ADR-0010
C	TAL VEZ	Evaluado únicamente como referencia académica de bajo nivel; sin aplicación directa en el alcance del MVP.	ADR-0010
NestJS	TAL VEZ	Framework Node.js/TypeScript evaluado en Sprint 0; desplazado del núcleo de persistencia por ADR-0012 en favor de Serverpod/Supabase Postgres.	ADR-0010, ADR-0012
Spring Boot	MEJOR NO	Descartado para el servicio Java en favor de frameworks livianos / Java estándar para reducir consumo de memoria en contenedores.	ADR-0010
React	MEJOR NO	Descartado para la interfaz de usuario al consolidarse Flutter como solución multiplataforma única.	ADR-0010
C++	MEJOR NO	Descartado por sobrecarga en gestión de memoria y falta de drivers de calidad que lo justifiquen.	ADR-0010
JavaScript	MEJOR NO	Descartado como lenguaje no tipado en favor de Dart, C# y Java para garantizar contratos estrictos de datos.	ADR-0010

Cuadrante IV: Herramientas para el Proyecto

Herramienta	Anillo	Justificación y Criterio en MANI	ADR Relacionado
SonarQube	SÍ O SÍ	Herramienta oficial de SAST integrada en GitHub Actions. Aplica Quality Gates bloqueantes sobre el código de Flutter, Java y .NET.	ADR-0005
Docker	SÍ O SÍ	Estándar de contenerización para todos los microservicios y ambientes locales y cloud. Garantiza reproducibilidad absoluta.	ADR-0004
Prometheus	SÍ O SÍ	Motor de recolección continua de métricas operacionales de contenedores, nodos y servicios.	ADR-0006
Postman + Newman	SÍ O SÍ	Diseño, especificación y ejecución automatizada de pruebas funcionales de API y validación de contratos en el pipeline de CI.	ADR-0004, ADR-0015
Grafana	SÍ O SÍ	Cuadros de mando centralizados para la visualización de métricas de rendimiento y salud de la plataforma.	ADR-0006
Datadog	SÍ O SÍ	Plataforma de APM, centralización de logs distribuidos y despacho automatizado de alertas de incidentes hacia Jira.	ADR-0006
Kubernetes	SÍ O SÍ	Orquestador de contenedores requerido curricularmente (PROY-08). Gestiona escalado, auto-recuperación y balanceo en Azure.	ADR-0004, ADR-0010
Emuladores Móviles	SÍ O SÍ	Emuladores oficiales Android e iOS para validación funcional de interfaces en QA y desarrollo Frontend.	Gobierno QA
k6	SÍ O SÍ	Herramienta para pruebas de carga, concurrencia y estrés de endpoints críticos (despacho y mensajería en tiempo real).	ADR-0010, Matriz V1
IA: Claude / ChatGPT / Gemini	SÍ O SÍ	Asistentes de ingeniería autorizados bajo la política estricta de gobernanza de IA para aceleración técnica y documentación.	ADR-0009
OWASP ZAP	TAL VEZ	Herramienta de DAST para escaneos de seguridad dinámica en Testing. Se encuentra en proceso de calibración de scripts en CI.	ADR-0005, ADR-0010
QuickSight / PowerBI	TAL VEZ	Plataformas de Business Intelligence evaluadas para tableros de analítica de negocio futura de los tenants.	ADR-0010

3. Matriz Integral de Herramientas (Proceso y Producto)

A continuación se resume la matriz operativa unificada del proyecto MANI, clasificando cada herramienta por su naturaleza, etapa del ciclo de vida y responsable asignado.

Herramienta	Naturaleza	Etapas DevSecOps	Propósito Principal en MANI	Responsable Titular	Estado Operativo
Jira	Proceso	Plan	Backlog, gestión de sprints, épicas, estimaciones y tableros Scrum/Kanban.	Nicolás León (PO) / Sara Albarracín (SM)	Adoptada
GitHub	Proceso / Producto	Code	Control de código fuente, Pull Requests, ADRs en Markdown y C4.	Daniel Ávila (DevOps)	Adoptada
OneDrive	Proceso	Governance	Documentación formal administrativa, actas, balances y hojas de cálculo Excel.	Sara Albarracín (SM)	Adoptada
Discord	Proceso	Plan / Daily	Canales de comunicación en tiempo real, daily escrita y coordinación.	Sara Albarracín (SM)	Adoptada
GitHub Actions	Proceso	Build / Deploy	Automatización de flujos de CI/CD, Quality Gates y empaquetado Docker.	Daniel Ávila (DevOps)	Adoptada
Flutter / Dart	Producto	Code	Interfaz móvil multiplataforma para clientes y aliados (Repo A).	Camila Beltrán / Nicolás Álvarez	Adoptada
Java (Maven)	Producto	Code / Build	Microservicio backend de reglas de negocio transaccionales (Repo B).	Juan Sebastián Álvarez / Daniel Ávila	Adoptada
.NET (C#)	Producto	Code / Build	Microservicio backend para módulos de alto rendimiento (Repo C).	Daniel Ávila / Juan Sebastián Álvarez	Adoptada
Supabase / Postgres	Producto	Data / Security	Persistencia relacional, RLS multi-tenant, Supabase Storage y Auth.	Juan Sebastián Álvarez	Adoptada
Docker	Proceso / Producto	Build / Package	Contenerización estándar de microservicios y ambientes.	Daniel Ávila (DevOps)	Adoptada
Kubernetes (AKS)	Producto	Operate / Scale	Orquestación, escalado automático y resiliencia en Azure.	Daniel Ávila (DevOps)	Adoptada
Postman / Newman	Proceso	Test	Pruebas funcionales de API y verificación automática de RLS en CI.	Santiago (QA)	Adoptada
k6	Proceso	Test (Perf)	Pruebas de carga, concurrencia determinista y latencia de endpoints.	Santiago (QA)	Adoptada
SonarQube	Proceso	Security (SAST)	Análisis estático de vulnerabilidades, code smells y Quality Gate.	Daniel Ávila / Santiago	Adoptada
OWASP ZAP	Proceso	Security (DAST)	Análisis dinámico de seguridad sobre el ambiente de Testing.	Santiago (QA)	En Calibración
Prometheus	Proceso / Producto	Monitor	Recolección continua de métricas operacionales de contenedores.	Daniel Ávila (DevOps)	Adoptada
Grafana	Proceso	Monitor	Visualización de tableros operativos de salud de servicios.	Daniel Ávila (DevOps)	Adoptada

Herramienta	Naturaleza	Etapas DevSecOps	Propósito Principal en MANI	Responsable Titular	Estado Operativo
Datadog	Proceso / Producto	APM / Alertas	Trazas distribuidas, logs centralizados y retroalimentación a Jira.	Daniel Ávila (DevOps)	Adoptada
Figma	Proceso	UX / Design	Prototipos, mockups interactivos y diseño visual de componentes.	Nicolás León / Camila Beltrán	Adoptada
Claude / Codex / Gemini	Proceso	Cross-Dev	Aceleración asistida de código, pruebas y documentación técnica.	Todo el Equipo	Regulada (ADR-0009)

4. Políticas y Lineamientos de Ingeniería (V2)

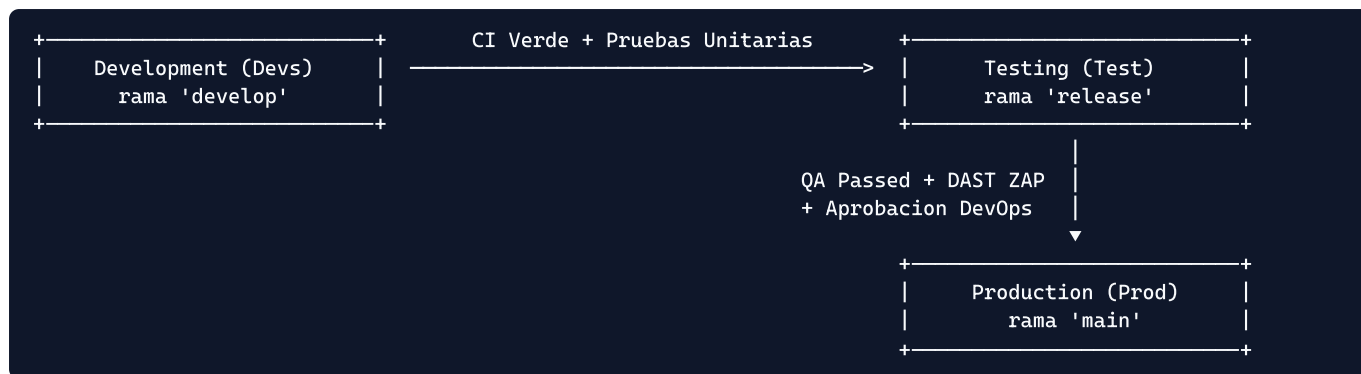
4.1 Política de Control de Versiones y Gestión de Ramas (Gitflow)

Conforme a **ADR-0004**, todo repositorio de MANI implementa estrictamente el modelo **Gitflow**:

- Ramas Principales:**
- `main` : Representa el código en Producción. Solo recibe mezclas (*merges*) provenientes de ramas `release/*` o `hotfix/*` . Queda protegida contra escrituras directas.
- `develop` : Rama de integración continua. Refleja el trabajo acumulado para el siguiente sprint.
- Ramas Temporales de Trabajo:**
- `feature/US-XX-descripcion-corta` : Para nuevas historias de usuario. Nace de `develop` y se integra de vuelta en `develop` vía Pull Request.
- `fix/BUG-XX-descripcion` : Para corrección de defectos detectados en desarrollo o testing. Nace de `develop` y se reintegra en `develop` .
- `release/vX.Y.Z` : Rama de estabilización previa a producción. Se congela para pruebas finales de QA en el ambiente Testing y auditoría de seguridad.
- `hotfix/HOTFIX-XX-descripcion` : Corrección urgente de incidentes críticos en producción. Nace de `main` y se reintegra tanto en `main` como en `develop` .
- `spike/SP-XX-descripcion` : Ramas de exploración técnica. No requieren cumplir con la suite completa de producción, pero su código no se fusiona a `develop` directamente sin un PR formal y su ADR resultante.
- Reglas Vinculantes de Pull Requests (PR):**
- Prohibido cualquier `commit` directo a `develop` o `main` .
- Todo PR debe estar vinculado a un Issue de GitHub y su correspondiente clave de Jira en el título o descripción (ej. `[SCRUM-104] feat: implementacion login multi-tenant`).
- Requiere como mínimo **una aprobación obligatoria** de un revisor técnico del equipo.
- El pipeline de CI debe finalizar en **verde** (pruebas unitarias aprobadas, build exitoso).
- El Quality Gate de SonarQube debe encontrarse en estado **Passed**.

4.2 Política de Promoción de Ambientes y Despliegues

Siguiendo el principio *Build Once, Deploy Anywhere* (**ADR-0004**):



1. **Inmutabilidad del Artefacto:** La misma imagen de contenedor Docker compilada y probada en el pipeline de desarrollo es la que se promueve a Testing y posteriormente a Producción. Queda estrictamente prohibido recompilar código entre ambientes.
2. **Segregación de Ambientes:**
3. **Development:** Se actualiza automáticamente tras cada merge exitoso en `develop`.
4. **Testing:** Se actualiza al cortar una rama `release/*`. En este entorno se ejecutan las pruebas funcionales de Newman, validación de aislamiento multi-tenant y escaneos de OWASP ZAP.
5. **Production:** Solo se despliega tras la aprobación formal del **DevOps titular** (Daniel Ávila) con validación previa de **QA** (Santiago) y el visto bueno del **Product Owner** (Nicolás León).
6. **Manejo de Variables y Secretos:** Queda prohibido incluir configuraciones específicas de entorno dentro de las imágenes. Estas deben inyectarse en tiempo de ejecución mediante *ConfigMaps* y *Secrets* gestionados en Azure / Kubernetes y GitHub Secrets.

4.3 Política de DevSecOps, Seguridad y Pruebas de Aislamiento

4.3.1 Análisis Estático (SAST) con SonarQube (ADR-0005)

- **Quality Gate Vinculante:** Ningún Pull Request podrá integrarse si introduce vulnerabilidades catalogadas como **Blocker** o **Critical**.
- Se exige un umbral de deuda técnica menor a 5% y ausencia de secretos o credenciales expuestas en el código.
- Los fallos menores catalogados como *Minor* o *Info* se registran como deuda técnica en el backlog de Jira para su refactorización en futuros sprints.

4.3.2 Análisis Dinámico (DAST) con OWASP ZAP (ADR-0005)

- Se ejecutan escaneos dinámicos automatizados sobre la API desplegada en el ambiente de Testing.
- Las APIs de Java y .NET deben exponer de forma obligatoria su especificación OpenAPI/Swagger actualizada para que OWASP ZAP pueda indexar las rutas y verificar inyecciones SQL, Cross-Site Scripting (XSS) y configuraciones inseguras de cabeceras HTTP.

4.3.3 Pruebas Automatizadas de Aislamiento Multi-Tenant (ADR-0015)

- Como salvaguarda del driver crítico **RNF-01** (Aislamiento Multi-Tenant) y para evitar revisiones manuales aisladas:
- Todo Pull Request que modifique código de autenticación, políticas de Row-Level Security (RLS), endpoints de datos o esquemas relacionales ejecuta obligatoriamente en GitHub Actions una suite de **6 casos de prueba de acceso cruzado** automatizados en **Newman**:
- Lectura de registros de otro tenant (debe retornar 0 resultados o 403 Forbidden).
- Listado de recursos ajenos (debe mostrar únicamente los recursos del tenant autenticado).
- Intento de escritura en un tenant ajeno (debe ser rechazado por RLS).

- Intento de borrado de recursos de otro tenant (debe ser rechazado por RLS).
- Invocación de endpoints con tokens expirados o alterados.
- Validación de aislamiento de documentos KYC en Supabase Storage.
- Si cualquiera de estas pruebas falla, el pipeline se bloquea de inmediato.

4.3.4 Almacenamiento Seguro de Documentos KYC (ADR-0013)

- Los documentos de verificación de identidad de aliados (cédula, antecedentes, certificaciones) se almacenan en un **único bucket privado de Supabase Storage**.
- Cada archivo se almacena bajo una ruta estricta parametrizada: `ruta = tenant_id/aliado_id/documento.ext`
- El acceso se restringe mediante políticas RLS declarativas a nivel de base de datos sobre la tabla `storage.objects`, garantizando que ningún aliado ni tenant pueda consultar documentos ajenos, satisfaciendo **RNF-01** y **RNF-10**.

4.3.5 Gestión de Secretos y Dependencias

- **Prohibición de Secretos en Repositorio:** Queda prohibido commitar contraseñas, tokens JWT, claves de acceso cloud o cadenas de conexión en el repositorio.
- **Inyección de Secretos:** Toda credencial se administra a través de **GitHub Repository Secrets** y variables de entorno del servidor.
- **Auditoría de Dependencias:** El análisis continuo de librerías de terceros se apoya en SonarQube y las alertas automatizadas de GitHub Dependabot para mitigar vulnerabilidades en dependencias de Maven, NuGet y Pub.

4.4 Política de Observabilidad, Monitoreo y Gestión de Incidentes (ADR-0006)

1. **Estandarización de Logs:**
2. Todos los microservicios (Java, .NET y Serverpod) deben emitir registros en formato **JSON estructurado**.
3. Los campos mínimos obligatorios por cada log son: `timestamp` (ISO-8601), `level` (INFO, WARN, ERROR), `service_id` (Repo A, B o C), `trace_id` (identificador único para trazabilidad distribuida), `tenant_id` y `message`.
4. **Monitoreo Continuo:**
5. Prometheus recolecta métricas operativas de CPU, memoria, tasa de peticiones y latencia de endpoints cada 15 segundos.
6. Grafana centraliza los cuadros de mando para visualización en tiempo real.
7. **Gestión Automatizada de Incidentes (Feedback Loop):**
8. Datadog supervisa en tiempo real la tasa de errores HTTP 5xx y fallos de infraestructura en Producción.
9. Cuando se activa una alerta crítica (ej. latencia sostenida > 2s o tasa de errores > 3% en una ventana de 5 minutos), el webhook de Datadog genera automáticamente un Issue de tipo **Bug/Incidente** en el Backlog de Jira.
10. El Scrum Master y el DevOps titular notifican la alerta en el canal `#issues` de Discord y la resolución debe atenderse en un plazo máximo de **4 horas hábiles**.

4.5 Política de Gobernanza para el Uso de Inteligencia Artificial (ADR-0009)

Conforme a **ADR-0009**, se autoriza el uso de modelos y herramientas de Inteligencia Artificial generativa (**Claude**, **OpenAI Codex / ChatGPT**, **Google Gemini**, **GitHub Copilot**) en el equipo de MANI bajo los siguientes lineamientos estrictos:

1. Casos de Uso Autorizados:

2. Generación de código base o plantillas de microservicios (*boilerplate*).
3. Apoyo en la redacción y formato de documentación técnica y borradores de ADR.
4. Generación de scripts de automatización para pipelines o suites de prueba.
5. Creación de código fuente para diagramas en formato Mermaid (`.mmd`).
6. Exploración comparativa de alternativas arquitectónicas y trade-offs.

7. Restricciones y Prohibiciones Inviolables:

8. **Confidencialidad de Datos:** Queda terminantemente prohibido introducir credenciales, secretos, API keys, información confidencial del cliente o datos de personas reales en plataformas de IA externas.
9. **Responsabilidad y Autoría Humana:** Todo código, diagrama o documentación generado por una IA debe ser revisado, entendido y validado línea por línea por la persona que lo incorpora al repositorio. La responsabilidad del commit y de cualquier defecto introducido recae exclusivamente en el autor humano.
10. **Gobernanza Arquitectónica:** Ninguna decisión de arquitectura se considerará oficial por el mero hecho de haber sido recomendada o generada por una IA. Toda decisión técnica estructural debe ser llevada a la **Mesa de Arquitectura**, evaluarse formalmente frente a alternativas y plasmarse en un **ADR numerado**.

4.6 Política de Gestión Documental y Diagramas (ADR-0001, ADR-0007, ADR-0008)

1. **Repositorio Único de Documentación Técnica:** Conforme a **ADR-0007**, toda la documentación técnica oficial del proyecto se centraliza en el repositorio `MANI-docs` en formato **Markdown (.md)**. Se descartan Confluence o Google Drive como fuentes primarias.
2. **Estructura Oficial de Carpetas:**
3. `/ADR/` : Decisiones de arquitectura numeradas en formato `ADR-NNNN-titulo.md`.
4. `/Product/` : Especificación de producto, SRS, SAD, análisis de requerimientos y glosario.
5. `/Project/` : Gobierno del equipo, matriz de herramientas y actas de la mesa.
6. `/Entregas/` : Documentos y entregables consolidados por cada hito académico/curricular (`Entrega1` , `Entrega2` , `Entrega3`).
7. `/Diagramas/` : Repositorio central de diagramas de arquitectura (**ADR-0008**), subdividido en:
 - `/Diagramas/c4/` : Diagramas de contexto, contenedores y componentes en C4.
 - `/Diagramas/flujos/` : Diagramas de proceso, flujos de servicio y pipelines.
 - `/Diagramas/Tecnologías/` : Diagramas de tecnologías, arquitectura de herramientas y Tech Radar.
8. **Nomenclatura y Versionado de Diagramas:**
9. Los archivos de diagrama deben nombrarse con la convención: `[tipo]_[nombre-descriptivo]_v[N].[ext]` (ej. `diagrama_tecnologias_alto_nivel_v2.jpg`).
10. Todo diagrama elaborado en Mermaid debe almacenar su código fuente editable con extensión `.mmd` junto a su renderizado gráfico para permitir auditoría de diferencias (*diffs*) en Git.

4.7 Criterios de Calidad: Definition of Ready (DoR) y Definition of Done (DoD)

Definition of Ready (DoR) para Sprint 2

Una historia de usuario se considera lista para ser estimada e ingresada al Sprint Planning únicamente si cumple: - [] Descripción clara en formato estándar (*Como [actor], quiero [acción], para [beneficio]*). - [] Criterios de aceptación detallados en formato **BDD / Gherkin** (*Dado / Cuando / Entonces*). - [] Prioridad de negocio asignada por el Product Owner (Crítica,

Alta, Media, Baja). - [] Estimación de esfuerzo acordada por el equipo de desarrollo en puntos de historia (Fibonacci). - [] Dependencias técnicas identificadas y sin bloqueos por spikes abiertos.

Definition of Done (DoD) para Sprint 2

Un ítem de trabajo se considera terminado (*Done*) únicamente cuando satisface: - [] Criterios de aceptación funcionales verificados y aprobados por el Product Owner. - [] Código alojado en la rama correspondiente mediante Pull Request aprobado por pares. - [] Pipeline de CI/CD en estado exitoso (compilación sin errores). - [] Cobertura de pruebas unitarias cumplida y suite de pruebas de regresión en verde. - [] Quality Gate de **SonarQube** aprobado (0 fallos Blocker / Critical). - [] Suite de pruebas de aislamiento multi-tenant aprobada en Newman (cuando aplique a endpoints/datos). - [] Documentación técnica, contratos de API o diagramas actualizados en `/docs` si el cambio lo requirió. - [] Ítem de Jira actualizado con enlaces a PRs, evidencias de pruebas y marcado como *Done*.

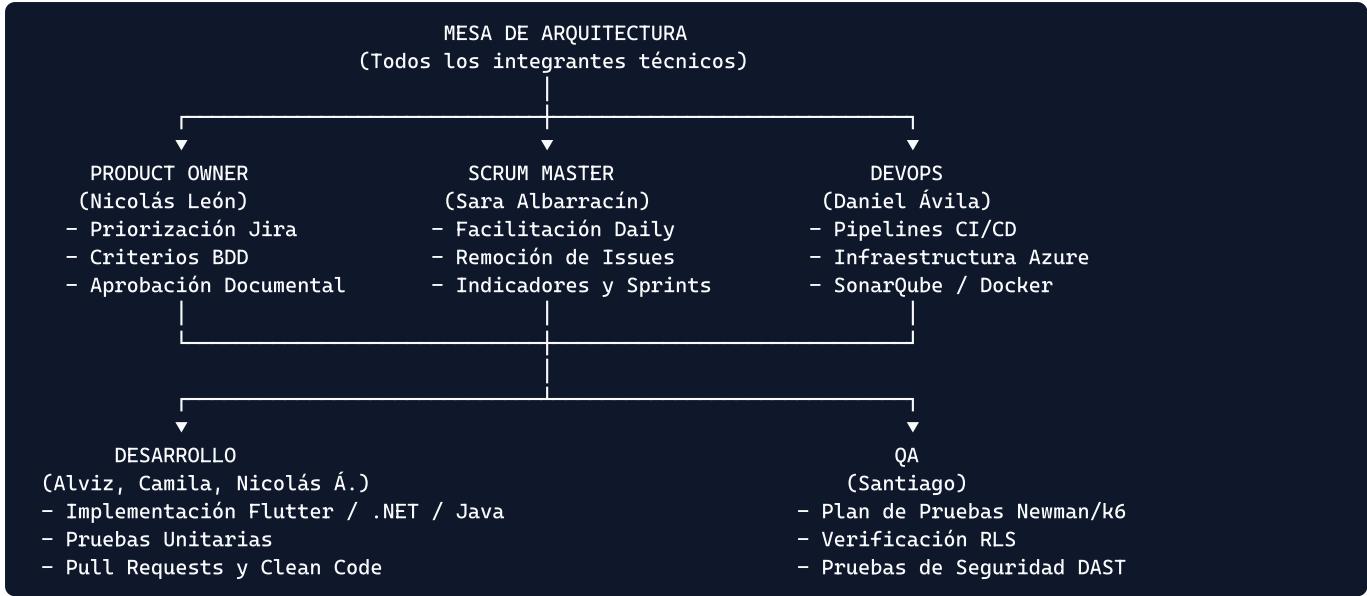
5. Evolución y Resolución de Inconsistencias (V1 → V2)

Durante la fase preliminar (V1 / Sprint 0) se identificaron diversas contradicciones y vacíos técnicos en el equipo. A continuación se presenta el estado de resolución formal alcanzado en esta versión V2:

#	Inconsistencia Identificada en V1	Estado en V2	Resolución Formal y Justificación
1	MongoDB vs. Row-Level Security (RLS) / Supabase Backend propuso MongoDB mientras QA requería RLS para RNF-01.	 Resuelto	Se descartó MongoDB y se ratificó PostgreSQL gestionado por Supabase con RLS nativo (ADR-0012). Se garantiza que el aislamiento multi-tenant vive a nivel de motor de datos y no de código.
2	Contradicción de Stack: Java/.NET vs. Dart/Serverpod ADR-0004 contemplaba Java y .NET por requisito curricular, mientras ADR-0012 planteaba backend puro en Dart.	 Resuelto	Se consolida la arquitectura multi-repositorio desacoplada evidenciada en el Diagrama de Alto Nivel: Repo A (Flutter/Dart en cliente/aliado conectado a BaaS Supabase), Repo B (Microservicio Java/Maven) y Repo C (Microservicio .NET), integrados mediante contratos REST/OpenAPI y eventos.
3	Duplicidad de Rastreadores (Jira vs. GitHub Projects vs. GitLab) Existían propuestas dispares sobre dónde gestionar el backlog.	 Resuelto	Se formalizó Jira como herramienta única y vinculante para el backlog del PO/SM (ADR-0002), sincronizada automáticamente con GitHub vía webhooks (ADR-0004). GitLab y Notion quedan descartados.
4	Docker Compose frente a Kubernetes Falta de claridad sobre necesidad de Kubernetes para el alcance del MVP.	 Resuelto	Kubernetes se ratifica como requisito curricular de infraestructura (PROY-08) sobre Azure AKS. Docker Compose se preserva exclusivamente para ambientes locales de desarrollo.
5	Vacío en Security Testing (SAST y DAST) En V1 no había herramientas definidas para seguridad continua.	 Resuelto	Se aprobaron e integraron formalmente SonarQube para SAST y OWASP ZAP para DAST en pipelines (ADR-0005).
6	Falta de Estrategia para Pruebas de Aislamiento Multi-Tenant Riesgo de fuga de datos entre empresas clientes.	 Resuelto	Se diseñó e implementó la suite automatizada de 6 casos en Postman/Newman dentro de GitHub Actions (ADR-0015).
7	Dispersión Documental y de Diagramas Documentos dispersos entre Confluence, Drive y Figma sin versionado.	 Resuelto	Centralización total en <code>/docs</code> en Markdown (ADR-0007) y estructura estricta para diagramas en <code>/docs/diagramas</code> (ADR-0008).
8	Uso no regulado de herramientas de IA Incertidumbre sobre límites éticos y técnicos al usar IA.	 Resuelto	Se ratificó la política vinculante de gobernanza de IA (ADR-0009), exigiendo revisión humana obligatoria y prohibición de subir credenciales o datos reales.

6. Roles y Responsabilidades del Equipo en el Ciclo de Herramientas

Para evitar solapamientos y asegurar la rendición de cuentas (*accountability*), se define la distribución operativa de responsabilidades:



7. Resumen de Aprobación y Trazabilidad

El presente **Documento de Herramientas, Políticas y Lineamientos V2** ha sido revisado por los roles líderes y aprobado de forma unánime por la Mesa de Arquitectura como línea base técnica para el Sprint 2:

Rol	Integrante	Responsabilidad en este Documento	Estado
Product Owner	Nicolás León	Aprobación de alcance, alineación con requerimientos de negocio y prioridades.	Aprobado
Scrum Master	Sara Albarracín	Aprobación de políticas de proceso, ceremonias, gestión de issues y gobierno.	Aprobado
DevOps Titular	Daniel Ávila	Aprobación de arquitectura de integración, CI/CD, infraestructura y DevSecOps.	Aprobado
QA Lead / Security	Santiago	Aprobación de estrategia de pruebas funcionales, k6, SonarQube, ZAP y RLS.	Aprobado
Frontend Leads	Camila Beltrán / Nicolás Álvarez	Aprobación de lineamientos de Flutter/Dart, UI/UX en Figma y emuladores.	Aprobado
Backend Lead	Juan Sebastián Álvarez (Alviz)	Aprobación de persistencia Supabase/Postgres, Serverpod y contratos de API.	Aprobado
Mesa de Arquitectura	Equipo Pleno (7 de 7 integrantes)	Ratificación técnica y cumplimiento de estándares de arquitectura.	Ratificado